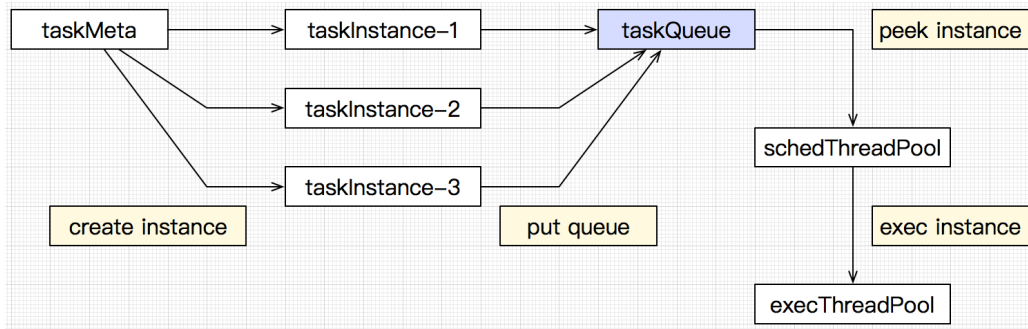


单机定时器的架构图



单机定时器的组成：

- taskMeta 表示任务元数据，包括任务描述、任务执行时间等。
- taskInstance 表示任务实例，记录真实执行的任务上下文。1 个 taskMeta 可以创建 N 个 taskInstance。
- taskQueue 表示任务队列，包含很多 taskInstance，按照创建时间有序。
- schedThreadPool 表示调度线程，读取 taskQueue，找到达到执行时间的 taskInstance，然后触发执行。
- execThreadPool 表示执行线程，执行 taskInstance，执行时间依赖 taskInstance 的运行时间。

## 常用的单机定时器

1、Java 的单机定时器包括 ScheduledExecutorService、Quartz 。

使用示例：

```
ScheduledExecutorService timer = Executors.newScheduledThreadPool(2);
System.out.println("当前时间 " + System.currentTimeMillis());
// 周期任务
timer.scheduleAtFixedRate(() ->
    System.out.println("task 执行一次 " + System.currentTimeMillis())
    , 1000, 2000, TimeUnit.MILLISECONDS);
```

输出：

```
当前时间 1744123125747
task 执行一次 1744123126913
task 执行一次 1744123128918
task 执行一次 1744123130918
task 执行一次 1744123132913
task 执行一次 1744123134914
```

2、Go 的单机定时器包括 Timer 单次任务、Ticker 周期任务。

使用示例：

```
func main() {
    ticker := time.NewTicker(1*time.Second)
    for range ticker.C {
        fmt.Println("触发 1 次 ", time.Now())
    }
}
```

输出：

|        |                                      |
|--------|--------------------------------------|
| 触发 1 次 | 2025-04-08 23:26:04.575489 +0800 CST |
| 触发 1 次 | 2025-04-08 23:26:05.574191 +0800 CST |
| 触发 1 次 | 2025-04-08 23:26:06.574151 +0800 CST |

## 手动实现 1 个单机定时器

### 1、定义任务元数据

任务元数据记录需要执行任务、触发时间等，贯穿任务的生命周期。

```
// 任务元数据
@Getter
@Setter
@Accessors(chain = true)
public class TaskMeta {
    Runnable bizTask;// 具体的任务。
    long delayMillis;// 任务初次延迟时间。毫秒。
    long intervalMillis;// 任务间隔时间。毫秒。
    long lastRunTimeMillis;// 最后一次运行时间。毫秒。
}
```

### 2、定义任务实例

任务元数据可以创建 1 或多个任务实例，任务实例的触发时间可以不相同。

任务实例放到任务队列，任务实例按照触发时间有序。

```
// 任务实例
@Getter
@Setter
@Accessors(chain = true)
public class TaskInstance implements Comparable<TaskInstance> {
    TaskMeta taskMeta;// 任务信息
    long runTimeMillis; // 运行时间。毫秒。

    // 排序。任务队列，按照运行实现做排序。
    @Override
    public int compareTo(TaskInstance o) {
        // 如果运行时间不相同，直接比较时间。
        if (this.runTimeMillis != o.runTimeMillis) {
            return (int) (this.runTimeMillis - o.runTimeMillis);
        }
        // 有可能运行时间相同。支持同时运行。
        return this.hashCode() - o.hashCode();
    }
}
```

### 3、定义单机定时器

属性包括状态码、任务队列、调度线程、执行线程等。

功能包括新增任务、新增任务实例、调度任务、执行任务等。

```

// 单机定时器。这里只展示关键部分。
public class SingleTimer {
    // 状态码。已经关闭。
    boolean beClosed = false;
    // 任务队列。
    TreeSet<TaskInstance> taskInstanceQueue = new TreeSet<>();
    // 队列的锁。并发安全。
    Lock queueLock = new ReentrantLock();
    // 条件变量。队列被改变了，触发调度
    Condition queueBeAdded = queueLock.newCondition();
    // 调度线程。这里只使用 1 个线程。
    ExecutorService schedThreadPool = Executors.newSingleThreadExecutor();
    // 执行线程。配置多个线程。
    ExecutorService execThreadPool = Executors.newFixedThreadPool(3);

    public SingleTimer() {
        schedThreadPool.execute(() -> handleTaskInstance());
    }

    // 单次运行任务。
    public void runTask(Runnable bizTask, long delayMillis) {
        this.runTask(bizTask, delayMillis, 0);
    }

    // 周期运行任务。
    public void runTask(Runnable bizTask, long delayMillis, long intervalMillis) {
        TaskMeta taskMeta = new TaskMeta().setBizTask(bizTask).setDelayMillis(delayMillis)
            .setIntervalMillis(intervalMillis).setLastRunTimeMillis(0);
        this.createTaskInstance(taskMeta);
    }

    // 创建任务实例。
    private void createTaskInstance(TaskMeta taskMeta) {
        long runTime = 0;
        // 如果没有运行过，就使用首次延迟时间。
        if (taskMeta.lastRunTimeMillis <= 0) {
            runTime = System.currentTimeMillis() + taskMeta.delayMillis;
        }
        // 如果不需要周期运行，就停止。
        else if (taskMeta.intervalMillis <= 0) {
            return;
        }
        // 周期运行。
        else {
            runTime = taskMeta.lastRunTimeMillis + taskMeta.intervalMillis;
        }

        // 生成 1 个实例。
    }
}

```

```

TaskInstance taskInstance = new TaskInstance().setTaskMeta(taskMeta)
    .setRunTimeMillis(runTime);
queueLock.lock();
try {
    taskInstanceQueue.add(taskInstance);
    queueBeAdded.signalAll(); // 通知调度线程
} catch (Exception e) {
    e.printStackTrace();
} finally {
    queueLock.unlock();
}
}

```

// 处理任务实例

```

private void handleTaskInstance() {
    // 循环处理。
    while (!beClosed) {
        this.handleTaskInstanceOnce();
    }
    System.out.println("退出");
    schedThreadPool.shutdown(); // 清理线程
    execThreadPool.shutdown(); // 清理线程
    queueLock.lock();
    try {
        taskInstanceQueue.clear(); // 清理队列
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        queueLock.unlock();
    }
}

```

// 处理任务实例

```

private void handleTaskInstanceOnce() {
    queueLock.lock();
    try {
        long waitMillis = 1000;
        // 有任务实例
        if (!taskInstanceQueue.isEmpty()) {
            // 判断开头任务，是否到时间。
            TaskInstance taskInstance = taskInstanceQueue.first();
            long now = System.currentTimeMillis();
            // 时间到了，执行任务。
            if (taskInstance.runTimeMillis <= now) {
                // 执行任务
                taskInstance = taskInstanceQueue.pollFirst();
                execThreadPool.execute(taskInstance.taskMeta.bizTask);
                taskInstance.taskMeta.lastRunTimeMillis = now;
                // 创建新任务
            }
        }
    }
}

```

```

        this.createTaskInstance(taskInstance.taskMeta);
        // 尝试继续调度。
        return;
    }
    // 还没有到时间，就等待。
    else {
        waitMillis = taskInstance.runTimeMillis - now;
    }
}
System.out.println(System.currentTimeMillis() + " 调度线程，wait=" + waitMillis);
// 等待有任务到来
queueBeAdded.await(waitMillis, TimeUnit.MILLISECONDS);
} catch (Exception e) {
    e.printStackTrace();
} finally {
    queueLock.unlock();
}
}
}
}

```

#### 4、测试

```

public class SingleTimerTest {
    public static void main(String args[]) throws Exception {
        // 单机定时器。
        SingleTimer timer = new SingleTimer();
        System.out.println(System.currentTimeMillis() + " 开始时间");

        // 单次任务。执行 1 次
        timer.runTask(() -> {
            System.out.println(System.currentTimeMillis() + " 单次任务 5-执行");
        }, 500);
        timer.runTask(() -> {
            System.out.println(System.currentTimeMillis() + " 单次任务 6-执行");
        }, 3000);
        // 周期任务。执行多次。
        timer.runTask(() -> {
            System.out.println(System.currentTimeMillis() + " 周期任务 A-执行");
        }, 1500, 2000);
        timer.runTask(() -> {
            System.out.println(System.currentTimeMillis() + " 周期任务 B-执行");
        }, 1500, 2000);

        Thread.sleep(7000); // 等待任务触发。
        timer.beClosed = true; // 关闭。
    }
}

```

#### 5、运行日志

2 个单次任务，分别触发 1 次，触发时间分别为 500ms、3000ms。

2 个周期任务，分别循环执行。首次触发时间为 1500ms，后续触发时间为 2000ms。

```
1743861649640  开始时间
1743861649640  调度线程, wait=1000
1743861649643  调度线程, wait=499
1743861649643  调度线程, wait=499
1743861649643  调度线程, wait=499
1743861649644  调度线程, wait=498
1743861650144  调度线程, wait=999
1743861650144  单次任务 5-执行
1743861651146  周期任务 A-执行
1743861651146  调度线程, wait=1497
1743861651146  周期任务 B-执行
1743861652644  调度线程, wait=501
1743861652644  单次任务 6-执行
1743861653145  调度线程, wait=1
1743861653145  周期任务 A-执行
1743861653147  调度线程, wait=1998
1743861653147  周期任务 B-执行
1743861655147  周期任务 A-执行
1743861655147  周期任务 B-执行
1743861655147  调度线程, wait=2000
退出
```

#### 引申：如果一个周期定时任务希望取消，怎么实现？

可以在 TaskMeta 加个属性 boolean beClosed，在创建 TaskInstance 时判断 beClosed，如果为 true，则不再创建新任务实例。

使用某些方式设置 beClosed。比如，新增定时任务时返回一个 future，实现 cancel 方法用于设置 beClosed。

取消定时任务是通用场景。比如，打电话过程中定时发送心跳消息，如果电话挂断，则取消心跳任务。

#### 引申：什么时机创建任务实例？

新增任务时，创建初始任务实例。因为需要任务实例去执行，类似于冷启动。

执行任务时，创建下一个任务实例。周期任务，不应该在新增任务时创建很多任务实例，因为执行次数无上限导致实例数量也没有上限。